hashlock.

Security Audit

Your Wallet (Crypto Wallet)

August 2025 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

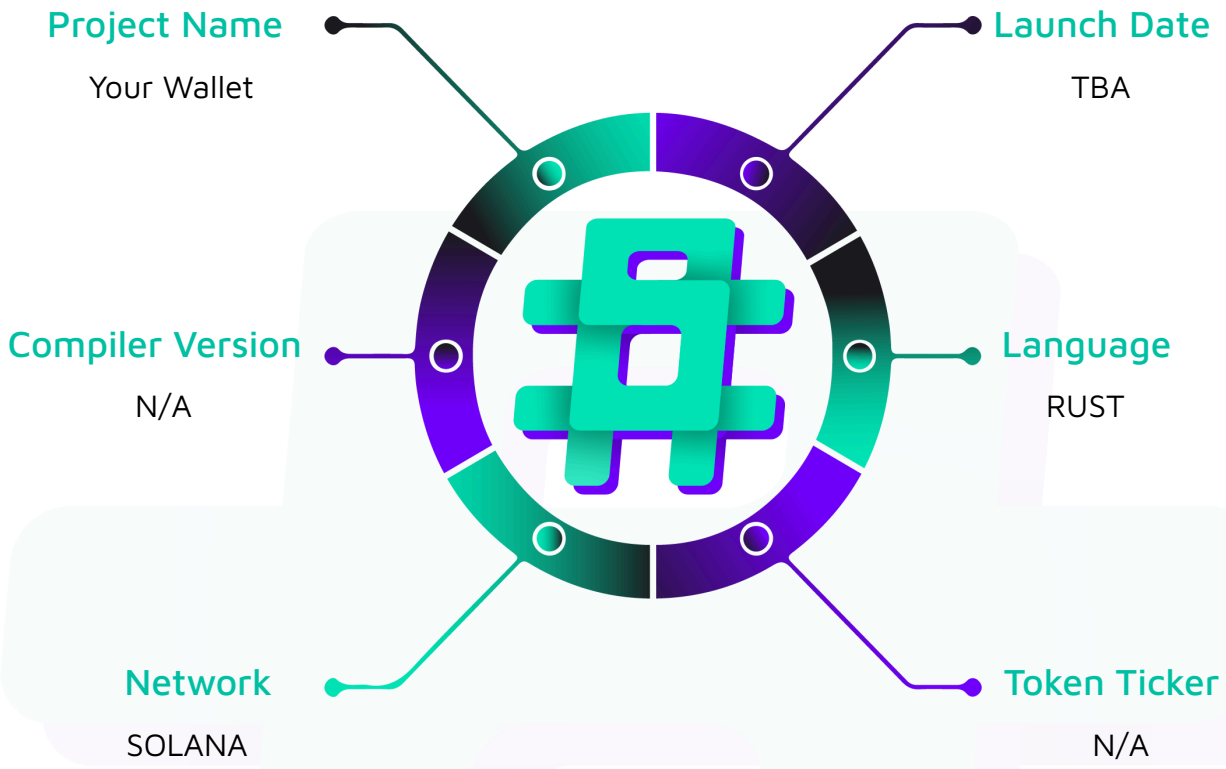
The Your Wallet team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

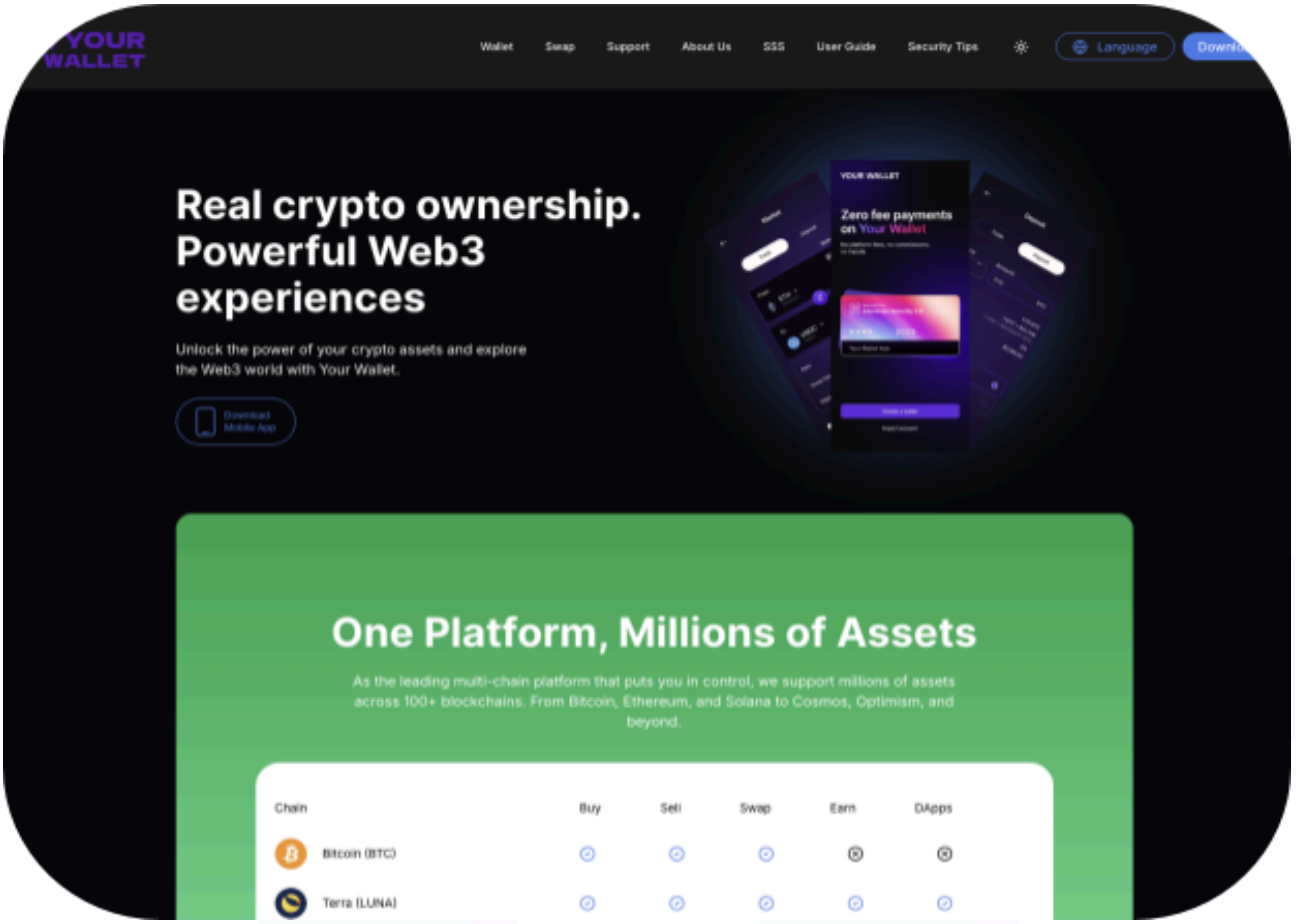
Your Wallet is a comprehensive mobile and desktop Web3 wallet designed to provide users with full control over their digital assets. It enables seamless buying, selling, swapping, and staking of cryptocurrencies. The platform emphasizes user privacy and security, offering features like encrypted backups, biometric authentication, and proactive alerts for risky transactions. Additionally, Your Wallet facilitates easy deposits from major exchanges and supports decentralized applications (dApps), making it a versatile tool for both crypto enthusiasts and developers seeking to engage with the Web3 ecosystem.

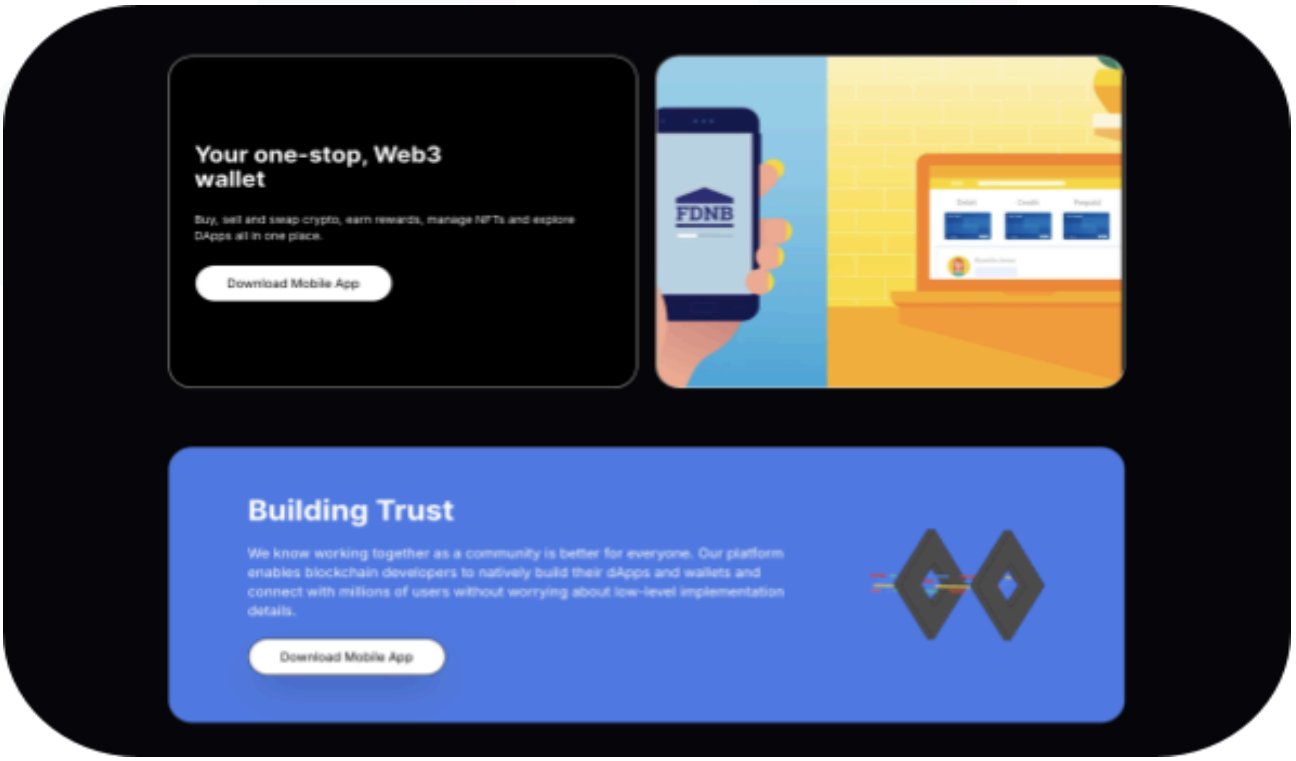
Project Name: Your Wallet
Project Type: Crypto Wallet
Compiler Version: ^0.8.42
Website: https://www.yourwallet.tr/
Logo:

Visualised Context:



Project Visuals:





Audit Scope

We at Hashlock audited the Rust code within the Your Wallet project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**Your Wallet Smart Contracts**
Platform	**Solana / Rust**
Audit Date	**August, 2025**
Contract 1	entrypoint.rs
Contract 2	error.rs
Contract 3	instruction.rs
Contract 4	lib.rs
Contract 5	processor.rs
Contract 6	state.rs
Audited GitHub Commit Hash	ce155f27e4960ddd3665e3edc80ddccde6b62c9b
Fix Review GitHub Commit Hash	cea5642fedf068472a0046bbe72d8eb92092c6fc

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

2 High severity vulnerabilities

2 Medium severity vulnerabilities

3 Low severity vulnerabilities

1 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
processor.rs - Allows users to: - Stake tokens into a selected pool during its active window. - Unstake to receive their tokens back, plus any earned rewards (subject to pool rules). - Allows admins to: - Create and initialize new staking pools with parameters (start/end times, min/max stake, reward rate) and seed them with rewards. - Add (top up) reward tokens to existing pools. - Set who controls minting for the token (rotate mint authority). - Set up a global manager that tracks the number of pools.	**Contract achieves this functionality.**

Code Quality

This audit scope involves the smart contracts of the Your Wallet project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Your Wallet project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] processor.rs - Misordered accounts in CPIs break unstake_tokens_from_pool

Description

The code passes uses the wrong positional order for the SPL Token-2022 CPIs. SPL Token matches accounts by index, so misordering the metas causes both `burn_checked` to fail at runtime.

Vulnerability Details

For `burn_checked`, the instruction expects `[account, mint, authority]`. The code has these misorders, which cause the program to attempt to unpack a token account from the executable `token_program` account and fail.

```
let burn_tokens_from_source = burn_checked(
    token_program.key,
    stake_pool_ata.key,
    token_mint.key,
    stake_pool.key,
    &[&stake_pool.key],
    amount,
    9,
)?;
invoke_signed(
    &burn_tokens_from_source,
    &[
```

```
        token_program.clone(),
        stake_pool.clone(),
        token_mint.clone(),
        stake_pool_ata.clone(),
        sysvar.clone(),
    ],
    &[&[
        b"stake_pool",
        pool.stake_pool_no.to_string().as_bytes(),
        &[pool.bump],
    ]],
)?;
```

Impact

Users cannot complete `unstake`. As the accounts are misordered, the program fails to find the correct token accounts and authority, leading to a failed instruction.

Recommendation

For `burn_checked` during unstake, pass `[stake_pool_ata, token_mint, stake_pool]`.

Status

Resolved

[H-02] processor.rs - Wrong amount recorded in TheStake

Description

When creating the user's stake record, the code writes the pool's cumulative total (`final_amount`) into `TheStake.stake_amount` instead of the user's credited amount. This corrupts the per-position principle.

```
// after applying the 1% haircut:

let credited = amount.amount - amount.amount/100;

let final_amount = credited + pool.total_tokens_staked;

pool.total_tokens_staked = final_amount;

let stake = TheStake {

    ...

    stake_amount: final_amount,    // BUG: should be `credited`

    ...

};
```

Vulnerability Details

Unstake logic later uses `stake.stake_amount` to compute both the principal returned and the reward:

```
entitled_reward = (stake.stake_amount / 1000) * pool.stake_reward;

amount_to_return = stake.stake_amount + entitled_reward;
```

If `stake_amount` holds the cumulative pool total, a later staker can withdraw far more than they actually staked.

Impact

Direct economic loss or denial of service. A user can over-withdraw and drain the pool, or unstakes will fail due to insufficient pool balance, locking funds until fixed. Rewards become nonsensical across users.

Recommendation

Record the per-user credited amount in `TheStake`, not the pool total.

Status

Resolved

Medium

[M-01] processor.rs - Hardcoded “new mint authority” pubkey string

Description

The function validates `new_authority` by parsing a literal string into a `Pubkey` and comparing it to the passed account. The string `"pda authority address"` is not a valid base58 public key, so `Pubkey::from_str(...).unwrap()` will panic at runtime, and the instruction will always fail.

```
let new_authority_address: Pubkey = Pubkey::from_str("pda authority address").unwrap();

if &new_authority_address != new_authority.key {

    return Err(InvalidAuthorityAddress.into());

}
```

Vulnerability Details

The code relies on a hardcoded placeholder instead of deriving or verifying the intended PDA. This creates a guaranteed panic today. Even if replaced with some static base58 value, the design would still be fragile and environment-specific, allowing misconfiguration or authority transfer to an unintended key.

Impact

Authority rotation is effectively denied.

Recommendation

Derive and enforce the PDA expected for the new authority and remove the literal string and `unwrap()`. Verify that the passed `new_authority` equals the derived PDA and then call `set_authority` to set `AuthorityType::MintTokens` to that PDA.

Status

Resolved

[M-02] processor.rs - Stake record not properly closed after unstake

Description

After unstake, the code only transfers lamports out of the stake account:

```
**the_stake.lamports.borrow_mut() -= value;

**staker.lamports.borrow_mut() += value;
```

It does not update the account's data or actually close the account. The stake record remains on-chain with its old fields intact.

Vulnerability Details

Leaving the stake record intact makes the on-chain state misleading. Any logic that scans for "open" stakes by reading records (e.g., non-zero amounts, timestamps, or missing closed flag) will still treat the record as active. Rent for an unused account is also not reclaimed.

Impact

Rent for an unused account is not reclaimed, and the dangling account can be accidentally reused or interacted with later, causing unexpected behavior and operational headaches.

Recommendation

After a successful unstake, physically close the account, after moving lamports out of a program-owned stake account (ideally a PDA), shrink and deassign:

```
the_stake.realloc(0, false)?; // shrink data to zero

invoke_signed(

    &solana_program::system_instruction::assign(

        the_stake.key, &solana_program::system_program::id()

    ),

    &[the_stake.clone()],
```

```
    stake_seeds, // seeds for the stake PDA if applicable
)?;
```

Status

Resolved

Low

[L-01] processor.rs - Missing validation of mint_authority_pda in add_tokens_to_reward_pool

Description

The function accepts a `mint_authority_pda` account and uses it to mint via `mint_to_checked` with `invoke_signed`, but never verifies that it equals the PDA derived from seed `"seed"` and `program_id`. In practice, an incorrect account will cause the CPI to fail closed due to a signature mismatch.

Recommendation

Derive and check the PDA early; fail fast on a mismatch, then use the same seeds for `invoke_signed`.

```
let (expected_pda, bump) = Pubkey::find_program_address(&[b"seed"], program_id);

if mint_authority_pda.key != &expected_pda {

    return Err(InvalidAuthorityAddress.into());

}

// proceed with mint_to_checked + invoke_signed using &[&[b"seed", &[bump]]]
```

Status

Resolved

[L-02] processor.rs - No mechanism to disable a pool

Description

Staking is gated by the `pool_active` flag, but there is no instruction exposed to change this flag after pool creation. Operationally, this means the pool cannot be disabled or paused once live.

The code enforces `pool.pool_active == 1` to allow staking, implying a pause/disable control exists. However, the program does not provide an admin/governance pathway to flip `pool_active` to `0` (or back to 1). The flag is therefore static in practice.

Recommendation

Add an admin-only instruction (or governance-controlled action) to toggle `pool_active` between enabled and disabled.

Status

Resolved

[L-03] processor.rs - Hardcoded account sizes for Pool and TheStake

Description

In `initialize_stake_pool`, the code uses `81` as the size when funding rent and creating the `stake_pool` account, which stores the `Pool` struct.

In `stake_tokens_to_pool`, the code uses `52` as the size when funding rent and creating the `the_stake` account, which stores the `TheStake` struct.

Recommendation

Compute the size instead of hardcoding it, and use the same computed value for both rent and `create_account`. Using `std::mem::size_of::<Pool>()` and `std::mem::size_of::<TheStake>()` is the safest approach.

Status

Acknowledged

QA

[Q-01] processor.rs - Duplicate signer/owner checks in `unstake_tokens_from_pool`

Description

The function validates `staker.is_signer`, and `stake_pool.owner == program_id` twice in succession. The second block is redundant and cannot improve safety because the first block already rejects invalid inputs and aborts execution.

Recommendation

Keep a single validation block early and remove the duplicate.

Status

Resolved

Centralisation

The Your Wallet project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the Your Wallet project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au